Prácticas de Ampliación de Sistemas Operativos • Validación

La utilidad `asosystest`

La utilidad `asosystest` permite probar un programa, por ejemplo, `tarea.c`, de manera sistemática y reproducible. Para hacerlo, se tiene que proporcionar un archivo de configuración como entrada con el conjunto de pruebas en formato JSON. Por defecto, `asosystest` ejecuta todas las pruebas descritas en el archivo de configuración. Sin embargo, se pueden ejecutar una o más pruebas arbitrarias con la opción `-t` seguida de una lista de números de test o rangos de números (p.e. 2-5) separados por comas. Un ejemplo podría ser `-t 1,3-5,7` para ejecutar los tests 1, 3, 4, 5 y 7. La opción `-d` proporciona información detallada sobre la ejecución de los tests.

```
$ ./asosystest.py -h
asosystest.py: Version: v0.23.1
usage: asosystest.py [-h] [options]

asosystest testing module v0.23.1.

options:
  -h, --help            show this help message and exit
  -i TEST_FILE, --in-test-file TEST_FILE
                        JSON file containing list of tests.
  -t TESTIDS, --testids TESTIDS
                        List of ranges of test IDs.
  -d, --debug           Enable debug mode.
  -s, --setup           Execute setup commands only.

Example: asosystest.py -i example.json -t 1,3-5,7
```

El formato del archivo de configuración es el siguiente:

```
{
    "setup": {
        "desc": "Archivo de configuración para validación de 'tarea'",
        "timeout": 1,
        "cmds": ["touch f1"],
        "binaries": ["tarea"]
    },
    "tests":
    [
        {
            "cmt": "Test existencia f1",
            "cmd": "ls f1",
            "out": [
                "f1\n"
```

```
            ],
            "rc": 0
        },
        {
            "cmt": "Test salida 'tarea'",
            "cmd": "./tarea # Argumentos",
            "out": [
                "Hello, ASO!\n"
            ],
            "rc": 0
        }
    ]
}
```

La sección `setup` contiene la configuración para todas las pruebas:

- `timeout`: Límite de tiempo en segundos para cada prueba. Si la prueba no ha terminado pasado este tiempo, se interrumpe considerándose fallida.
- `cmds`: Comandos a ejecutar antes del inicio de las pruebas.
- `binaries`: Binarios necesarios para llevar a cabo las pruebas.

La sección `tests` es una lista de pruebas con los siguientes campos:

- `cmt`: Comentario(s).
- `cmd`: Línea de órdenes objeto de la prueba. Si se especifican varias líneas, éstas se ejecutan secuencialmente.
- `out`: Salida esperada en `stdout` y/o `stderr`.
- `rc`: Código de retorno esperado.
- `timeout` (opcional): Límite de tiempo en segundos para esta prueba.

Para ejecutar las pruebas de la sección `tests`, la utilidad `asosystest` crea un directorio temporal durante el procesamiento de la sección `setup`. De este modo, cualquier archivo creado durante la ejecución de las pruebas, se elimina cuando finaliza `asosystest`.

Validación de `tarea.c` con `asosystest`

Para ejecutar todas las pruebas del archivo de configuración `tarea.json`:

```
$ ./asosystest.py -i tarea.json
asosystest.py: Version: v0.23.1
asosystest.py: Created temporary directory: '/tmp/tmpba4mt1i9'.
asosystest.py: Successfully copied binaries: '['tarea']'.
asosystest.py: Successful os.chdir('/tmp/tmpba4mt1i9').
asosystest.py: Successfully executed setup commands '['touch f1']'.
asosystest.py: Archivo de configuración para validación de 'tarea'.T01: Result   :
OK!
asosystest.py: Archivo de configuración para validación de 'tarea'.T02: Result   :
OK!
asosystest.py: Successful tests: [1, 2]
```

Si modificamos `tarea.c` para que imprima `Hello, ASO!!!`:

```
$ ./asosystest.py -i tarea.json
asosystest.py: Version: v0.23.1
asosystest.py: Created temporary directory: '/tmp/tmpa469mivk'.
asosystest.py: Successfully copied binaries: '['tarea']'.
asosystest.py: Successful os.chdir('/tmp/tmpa469mivk').
asosystest.py: Successfully executed setup commands '['touch f1']'.
asosystest.py: Archivo de configuración para validación de 'tarea'.T01: Result   :
OK!
asosystest.py: Archivo de configuración para validación de 'tarea'.T02: Result   :
ERROR!
asosystest.py: Successful tests: [1]
asosystest.py: Failed tests: [2]
```

Para obtener más información acerca del error:

```
$ ./asosystest.py -i tarea.json -t 2 -d
asosystest.py: Version: v0.23.1
asosystest.py: Created temporary directory: '/tmp/tmpvxnn7env'.
asosystest.py: Successfully copied binaries: '['tarea']'.
asosystest.py: Successful os.chdir('/tmp/tmpvxnn7env').
asosystest.py: Successfully executed setup commands '['touch f1']'.

asosystest.py: Archivo de configuración para validación de 'tarea'.T02: Result   :
ERROR!
asosystest.py: Archivo de configuración para validación de 'tarea'.T02: Command  :
'./tarea # Argumentos'
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
asosystest.py: Archivo de configuración para validación de 'tarea'.T02: Expected
(rc 0): ['H', 'e', 'l', 'l', 'o', ',', ' ', 'A', 'S', 'O', '!', '-->', '\n']
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
asosystest.py: Archivo de configuración para validación de 'tarea'.T02: Produced
(rc 0): ['H', 'e', 'l', 'l', 'o', ',', ' ', 'A', 'S', 'O', '!', '-->', '!', '!',
'\n']
asosystest.py: Failed tests: [2]
```

Como podemos ver, la salida de `asosystest` nos indica con `-->` cuál es el primer carácter en el que la salida esperada y la salida producida discrepan. En este caso, vemos que en lugar de un `\n`, que era lo esperado, se ha recibido un `!!\n`.

La utilidad `asosystest` también verifica que el código de retorno es correcto. Por ejemplo, si sustituimos `exit(EXIT_SUCCESS)` por `exit(EXIT_FAILURE)` en `tarea.c`:

```
$ ./asosystest.py -i tarea.json -t 2 -d
asosystest.py: Version: v0.23.1
asosystest.py: Created temporary directory: '/tmp/tmpivuwgukx'.
asosystest.py: Successfully copied binaries: '['tarea']'.
```

```
asosystest.py: Successful os.chdir('/tmp/tmpivuwgukx').
asosystest.py: Successfully executed setup commands '['touch f1']'.

asosystest.py: Archivo de configuración para validación de 'tarea'.T02: Result   :
ERROR!
asosystest.py: Archivo de configuración para validación de 'tarea'.T02: Command  :
'./tarea # Argumentos'
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
asosystest.py: Archivo de configuración para validación de 'tarea'.T02: Expected
(rc 0): ['H', 'e', 'l', 'l', 'o', ',', ' ', 'A', 'S', 'O', '!', '\n', '-->']
asosystest.py: Archivo de configuración para validación de 'tarea'.T02:
asosystest.py: Archivo de configuración para validación de 'tarea'.T02: Produced
(rc 1): ['H', 'e', 'l', 'l', 'o', ',', ' ', 'A', 'S', 'O', '!', '\n', '-->']
asosystest.py: Archivo de configuración para validación de 'tarea'.T02: Return
code mismatch [0 != 1]!
asosystest.py: Failed tests: [2]
```

Validación de `minigrep` y `watchdir`

Cada programa tiene su archivo de configuración. Se ejecutan igual que el de `tarea.json`, con `-i`, y admiten `-t` y `-d`:

```
$ ./asosystest.py -i minigrep.json            # Obligatoria (36 pruebas)
$ ./asosystest.py -i minigrep.opcional.json   # Ampliaciones -n y -o (12)
$ ./asosystest.py -i watchdir.json            # Obligatoria (30 pruebas)
```

Los binarios `minigrep` y `watchdir` se compilan con `make`, y `asosystest.py` los copia al directorio temporal, igual que `tarea`. Los archivos `minigrep.json` y `watchdir.json` contienen las pruebas **obligatorias**: hay que pasarlas todas para aprobar. El archivo `minigrep.opcional.json` contiene las pruebas de las ampliaciones **optativas** `-n` y `-o` de `minigrep`, que se describen en `Tarea_minigrep.md`. Están en un archivo aparte para que, si no las implementas, no veas fallos en la batería obligatoria.

El comentario (`cmt`) de cada prueba de `minigrep.opcional.json` indica a qué ampliación pertenece:

- `-n:` pruebas de la opción `-n`.
- `-o:` pruebas de la opción `-o`.
- `-n -o:` pruebas de las dos opciones juntas. Solo tienen sentido si implementas ambas.

Una ampliación solo cuenta como hecha si pasan **todas** sus pruebas. Fallar pruebas de este archivo no suspende, pero resta 1 punto a la nota de `minigrep` por cada ampliación que no las pase todas. Para ver qué ampliación falla y por qué, ejecuta solo sus pruebas con el detalle del error, por ejemplo, las pruebas 1 a 5 de `-n`:

```
$ ./asosystest.py -i minigrep.opcional.json -t 1-5 -d
```

Las pruebas de tamaño grande de `minigrep.json` trabajan con un archivo de 1 GB, que se genera en `/tmp/1GB.txt` la primera vez y se reutiliza después mientras exista. Por eso esas pruebas tardan más. Si lo

borras, se vuelve a crear.

Las utilidades `genera_bytes.py` y `genera_lineas.py`

La utilidad `genera_bytes.py` produce un flujo de bytes aleatorios, mientras que la utilidad `genera_lineas.py` crea una secuencia de líneas de texto aleatorias de tamaño arbitrario. Estas utilidades se emplean para generar la entrada necesaria para validar las tareas.

Ambas utilidades deben tener permisos de ejecución tras su descarga del Aula Virtual. De lo contrario, la ejecución de `asosystest.py` fallará mostrando el error: `asosystest.py: Error: Binary 'genera_bytes|lineas.py' not found.`.

La biblioteca `libasosysintercept.so.0.1`

El archivo `asosysintercept.c` contiene el código fuente de la biblioteca compartida `libasosysintercept.so.0.1`. Esta biblioteca permite capturar llamadas al sistema y funciones de biblioteca para alterar su comportamiento:

- Inyectando errores, por ejemplo, haciendo que una llamada a `write()` devuelva `-1` y actualice el valor de `errno`.
- Forzando la realización de lecturas o escrituras parciales cuando se llama a `read()` y `write()`, respectivamente.
- Deshabilitando ciertas funciones de biblioteca o llamadas al sistema como, por ejemplo, `strcat`.

Nota: La lista completa de funciones de biblioteca y llamadas al sistema deshabilitadas se puede encontrar en la sección `NOOP` del archivo fuente `asosysintercept.c`. La familia de `printf` no está en esa lista: `snprintf()` puede usarse, por ejemplo, para convertir en texto el número de línea de la opción `-n` de `minigrep`.

Para compilar la biblioteca, hay que usar la utilidad `make`:

```
$ make libasosysintercept.so.0.1
cc -ggdb3 -Wall -Werror -Wno-unused -std=c11 -fPIC -pie   -c -o asosysintercept.o
asosysintercept.c
cc --shared -fPIC -Wl,--no-as-needed -ldl asosysintercept.o -o
libasosysintercept.so.0.1
```

Como se puede comprobar, la compilación de la biblioteca requiere opciones distintas a las usadas para compilar los ejemplos, `minigrep.c` o `watchdir.c`. Por tanto, la compilación de `asosysintercept.c` fallará con la configuración de compilación de `.vscode/launch.json` de la copia de Visual Studio Code disponible en el Aula Virtual:

```
Iniciando la compilación...
/usr/bin/gcc -ggdb3 -Wall -Werror -Wno-unused -std=c11 /path/to/asosysintercept.c
-o /path/to/asosysintercept
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/11/../../../x86_64-linux-gnu/Scrt1.o:
en la función `_start':
(.text+0x1b): referencia a `main' sin definir
```

```
collect2: error: ld returned 1 exit status

La compilación ha finalizado con errores.

 *  El proceso del terminal finalizó con el código de salida -1.
 *  Las tareas reutilizarán el terminal, presione cualquier tecla para cerrarlo.
```

Para usar la biblioteca, basta con anteponer al binario `LD_PRELOAD=./libasosysintercept.so.0.1`, tal y como puede verse en los archivos de configuración `minigrep.json`, `minigrep.opcional.json` y `watchdir.json`. Para más detalles, véase Linux man pages online • System Manager's Manual • ld.so.

Requerimientos

Las utilidades `asosystest.py`, `genera_bytes.py` y `genera_lineas.py` descritas en el presente documento requieren **Python >= 3.10**. Nótese que en los laboratorios, Ubuntu 22.04 LTS incluye Python 3.10.12.